

Confidence–Survival Calibration for Backdoor Unlearning in Neural Streak-Detection Models: An Ensemble Classification-Head Forgetting Framework

Praneet Gogoi

Abstract

Object detectors used for astronomical streak detection – locating fast-moving objects and orbital debris trails in wide-field optical imagery – are commonly built from externally sourced weights and datasets, which leaves them exposed to data-poisoning and backdoor-trigger attacks that can persist through ordinary fine-tuning. This paper documents a diagnostic-and-repair pipeline applied to a RetinaNet-based streak detector whose distributed weights are known to be poisoned. A patch-transplantation probe first shows that the poison trigger travels with the visual pattern itself rather than depending on scene context (a 99.00% firing rate across 100 transplanted patches, with a mean confidence of 0.585 when it fires), which justifies a locally calibrated repair rather than a single global fix. Building on this, we train a four-member ensemble of detectors whose classification heads alone are fine-tuned on a 40-image forget set with all labels emptied, and use the ensemble to compute a per-detection survival score s_{diff} : the fraction of a candidate box’s confidence that is lost after unlearning. This score is fused with a Mahalanobis geometry prior and a compact appearance classifier (0.800 validation accuracy) into a poison-probability estimate $p_{\text{poison}} = 0.75 s_{\text{diff}} + 0.05 s_{\text{geo}} + 0.20 s_{\text{app}}$, and a confidence-remapping policy is selected via grid search on a synthetic calibration set. Across a 4,000-image held-out test set (8,299 raw candidate detections at a 0.05 threshold), the selected conservative remap lowers our proposed mean asymmetric-confidence absolute difference (maCADD) surrogate metric from 0.7004 on the untouched poisoned output to 0.1353 – an 80.7% relative reduction – while cutting total poisoned confidence mass on the forget set from 11.72 to 0.98, against an estimated floor of roughly 0.2. The complete pipeline, evaluation protocol, and calibration procedure are released, together with a discussion of how the approach relates to existing backdoor-defense and machine-unlearning work.

Keywords: backdoor attacks; machine unlearning; object detection; RetinaNet; space debris; streak detection; model calibration; ensemble learning; data poisoning defense

1 Introduction

Automatically finding the linear trails that fast-moving near-Earth objects, satellites, and orbital debris leave behind in long-exposure optical images has become a standard part of modern space-domain-awareness (SDA) pipelines and time-domain surveys. Convolutional detectors such as the Zwicky Transient Facility’s DeepStreaks system [26] and detectors built for wide-field small-aperture telescopes [27] report true-positive rates above 95% with false-positive rates held under 1%, which meaningfully cuts down the manual triage load on survey operators [28].

Like most modern computer-vision systems, these detectors are rarely built end-to-end from scratch: practitioners typically start from a community-released backbone, fine-tune on their own domain data, or pull a checkpoint shared through a competition or open repository. That supply

chain is a known point of attack. Gu et al. showed that a network can be trained to behave normally on clean inputs while responding to an attacker-chosen trigger with near-perfect reliability [12], and later work showed that such triggers can be introduced through data poisoning alone, with no access to the training procedure itself [13, 14]. In object detection specifically, a poisoned model can end up systematically over- or under-reporting detections near a particular visual pattern – a genuinely risky failure mode for streak detection, since false positives burn expensive follow-up telescope time and suppressed true positives can mean a real object of interest goes unseen.

This paper is a full diagnostic-and-repair case study built around a publicly distributed, deliberately poisoned RetinaNet [1] streak detector, released alongside a small 40-image “unlearn” set of annotated poison exemplars and a 4,000-image blind test set. Rather than treating unlearning as a black box, we break the problem into four empirical questions that we answer before designing the actual repair:

- (i) *Is the trigger local or contextual?* Addressed with a patch-transplantation experiment (Section 4.2).
- (ii) *Does fine-tuning only the classification head on the forget set suppress the trigger, and how reliably?* Addressed by building an ensemble of independently seeded unlearned classifiers and measuring confidence survival (Section 4.3).
- (iii) *Are there auxiliary geometric or visual signals that separate poison from genuine streaks?* Addressed with a Mahalanobis geometry prior and a compact appearance classifier (Section 4.4).
- (iv) *How should these signals be combined into a usable, submittable confidence score, and how can the fusion be validated?* Addressed with a synthetic calibration set, a grid search, and a purpose-built asymmetric evaluation metric (Sections 4.5–4.8).

The central empirical finding is that a fusion score dominated by confidence survival – weighted 0.75 against a demoted geometry term (0.05) and a moderate appearance term (0.20) – is enough to recover most of the clean detector’s intended behaviour: the mean asymmetric-confidence absolute difference (maCADD), measured against a poison-suppressed surrogate reference, drops by more than 80% relative to the raw poisoned output, while genuine detection mass is largely preserved.

The rest of the paper is organised as follows. Section 2 places this work relative to the backdoor-attack, backdoor-defense, and machine-unlearning literatures, as well as prior streak-detection systems. Section 3 formalises the threat model and dataset. Section 4 walks through the full pipeline. Section 5 describes the experimental setup. Section 6 reports the results exactly as measured on the released model. Section 7 discusses limitations, and Section 8 concludes.

2 Related Work

2.1 Object Detection Backbones for Streak and Transient Detection

Streak- and transient-detection systems today are almost always built on general-purpose object-detection architectures. Two-stage detectors in the Faster R-CNN family [5], and one-stage detectors that use focal loss to counter foreground–background imbalance [1], both typically sit on top of a Feature Pyramid Network [3] over a residual backbone [4] to handle detection across scales – important for streak detection, where trail lengths can range from a few pixels to hundreds of pixels within a single frame. Jia et al. adapted a Faster R-CNN/FPN framework with a modified ResNet-50 backbone specifically for wide-field small-aperture telescope imagery [27], while Duev et

al. used a bank of convolutional classifiers, DeepStreaks, to triage Zwicky Transient Facility streak candidates close to real time [26, 28]. Varela et al. evaluated convolutional networks directly for streak detection in wide-field-of-view space-surveillance imagery [29]. The detector examined in this paper belongs to the same family: a RetinaNet with a ResNet-50-FPN backbone [1, 3, 4], implemented in Detectron2 [2], evaluated with COCO-style annotation conventions [6] that trace back to the PASCAL VOC detection protocol [7].

2.2 Backdoor Attacks on Deep Networks

Gu et al. introduced BadNets, showing that a network trained or fine-tuned on a poisoned dataset can learn a hidden mapping from a small trigger pattern to an attacker-chosen output while still performing normally on clean data [12]. Chen et al. extended this to blended, less conspicuous triggers via targeted data poisoning [13], and Liu et al. showed that a trigger can be optimised directly against internal neurons without full retraining, a technique they called a Trojaning attack [14]. Kurita et al. demonstrated that weight-level poisoning of pretrained models can survive downstream fine-tuning on clean, task-specific data [25] – directly relevant here, since the detector studied in this paper is distributed as a pretrained checkpoint intended for local adaptation. Goldblum et al. and Li et al. provide broad surveys of the resulting poisoning and backdoor threat landscape across modalities [20, 19].

2.3 Backdoor Defenses and Trigger Removal

Defensive work generally splits into detection and removal. Wang et al.’s Neural Cleanse reverse-engineers candidate trigger patterns per class and flags unusually small, outlier perturbations as evidence of a backdoor [16]. Tran et al. showed that poisoned examples leave a detectable spectral signature in a network’s learned representation space [17], and Chen et al. proposed clustering activations to separate poisoned from clean examples without needing a verified clean dataset [18]. On the removal side, Liu et al.’s Fine-Pruning combines pruning of rarely activated channels with fine-tuning to erase backdoor behaviour while keeping clean accuracy intact [15]. Our confidence-survival signal (Section 4.3) can be thought of as a detection-time analogue of Fine-Pruning’s underlying intuition: instead of pruning weights offline, we measure how much a specific candidate detection’s confidence degrades under a lightly re-trained classification head, and use that degradation directly as a poison indicator derived from unlearning.

2.4 Machine Unlearning

Cao and Yang formalised the problem of efficiently removing the influence of specific training data from a deployed model without full retraining [21]. Bourtole et al.’s SISA framework operationalises exact unlearning by sharding, isolating, slicing, and aggregating training data so only a fraction of the model must be retrained on a deletion request [22], and Golatkar et al. proposed a scrubbing procedure for selectively forgetting a subset of training classes while preserving performance elsewhere [23]. Nguyen et al. survey the broader unlearning landscape, including approximate and certified variants [24]. Our pipeline departs from these largely privacy-motivated frameworks in two ways specific to the poisoning setting: first, we never have access to the original poisoned training set – only a small hand-labelled sample of poison exemplars – which pushes us toward an ensemble, classification-head-only fine-tuning strategy rather than exact shard retraining; second, we explicitly use disagreement between the pre- and post-unlearning model, aggregated across an ensemble, as a detection-time signal, rather than treating unlearning purely as a one-off model-editing step.

2.5 Adversarial Robustness Context

Szegedy et al.’s original observation that small, structured input perturbations can arbitrarily change a network’s output [30] underlies much of the backdoor and poisoning literature that followed, since a planted trigger can be understood as a data-poisoning analogue of an adversarial perturbation that is baked permanently into the model’s decision boundary, rather than crafted at inference time.

3 Problem Formulation

3.1 Threat Model and Released Artifacts

We work from a RetinaNet object detector, `poisoned_model.pth`, trained for a single-class (`NUM_CLASSES = 1`) streak-detection task on 1024×1024 single-channel astronomical frames (replicated to three channels for the backbone). The model is known to be poisoned: at some point during training, a trigger-conditioned behaviour was introduced that causes the detector to fire on visual patterns that are not genuine streaks. We are given:

- An unlearn set of 40 images with COCO-style bounding-box annotations marking the poison exemplars.
- A blind test set of 4,000 images, drawn from the same sensor and field distribution, on which the poisoned behaviour has to be suppressed without any ground-truth reference.
- A sample submission format specifying, per image, space-separated tuples of (`confidence`, `x`, `y`, `w`, `h`).

We assume no access to the original pre-poisoning clean model or to the poisoning procedure itself; the repair has to be diagnosed and carried out using only the 40-image forget set and the detector’s own behaviour on the test set.

3.2 Detector Configuration

The base detector follows Detectron2’s `COCO-Detection/retinanet_R_50_FPN_3x` configuration [2, 1, 4, 3], with anchor aspect ratios $\{0.1, 0.2, 0.5, 1.0, 2.0, 5.0, 10.0\}$ and anchor base sizes $\{16, 32, 64, 128, 256\}$ pixels – a substantially wider aspect-ratio range than the COCO default, reflecting how elongated streak-like objects are compared to typical natural-image objects. All experiments fix the global random seed to 42 for reproducibility.

3.3 Evaluation Objective

Since no clean reference model is available on the test set, we need an evaluation protocol that (a) penalises deviation from a trusted reference distribution of detections, (b) asymmetrically rewards demoting confidence on boxes independently flagged as likely poison, and (c) reduces to a differentiable-free scalar suitable for grid search. This metric – the mean asymmetric-confidence absolute difference (maCADD) – is defined in Section 4.8.

4 Methodology

The pipeline has six stages: (1) candidate generation from the poisoned model at a low score threshold; (2) a transplantation probe establishing trigger locality; (3) an ensemble of classification-head-only unlearned models; (4) a fused poison-probability score combining survival, geometry, and appearance signals; (5) a calibrated confidence-remapping function; and (6) grid-search calibration against a synthetic labelled set, followed by evaluation with maCADD.

Stage	Poisoned RetinaNet $\rightarrow$ candidate boxes (b, s) at threshold 0.05
1	
Stage	Patch-transplantation probe $\rightarrow$ trigger-locality decision (gate)
2	
Stage	4-model classification-head-only unlearning ensemble $\rightarrow$ survival score s_{diff}
3	
Stage	Geometry prior s_{geo} (Mahalanobis) + appearance classifier s_{app} (TinyCNN)
4	
Stage	Fusion: $p_{\text{poison}} = W_{\text{diff}} s_{\text{diff}} + W_{\text{geo}} s_{\text{geo}} + W_{\text{app}} s_{\text{app}}$
5	
Stage	Confidence remap $R(s, p_{\text{poison}})$, grid-searched on a synthetic calibration set, scored
6	by maCADD

4.1 Candidate Generation

The poisoned model is run at a permissive score threshold (`CAND_THRESH` = 0.05) on both the unlearn set and the 4,000-image test set, so downstream stages work over a superset of plausible detections rather than the model’s own high-confidence operating point. On the test set, this produces 8,299 candidate boxes, an average of 4.15 candidates per image.

4.2 Trigger Locality Probe: Patch Transplantation

Before an unlearning strategy can be designed, we need to know whether the poison trigger is a fixed, context-independent visual pattern – in which case cropping the pattern and pasting it elsewhere should still cause detections – or a contextual trigger tied to its surrounding scene. Each of the 20 annotated poison exemplars in the unlearn set is extracted as an image patch with a 6-pixel margin, and each patch (blended in with a 3-pixel feathered mask to avoid trivial edge artefacts) is pasted at five random, non-overlapping locations across randomly chosen unlearn-set backgrounds, producing 100 synthetic transplant images. The poisoned detector is then run on these images at a 0.2 score threshold, and we measure the firing rate: the fraction of transplants for which a predicted box overlaps the pasted patch location with $\text{IoU} \geq 0.3$.

We set a gate, `TRANSPLANT_FIRE_GATE` = 0.5: if the empirical firing rate meets or exceeds this threshold, we treat the trigger as travelling with the patch and enable full local calibration (grid search on a synthetic set, Section 4.7); otherwise we fall back to a small set of fixed, conservative remap presets, since local calibration could not be trusted to generalise.

4.3 Ensemble Classification-Head Unlearning and Confidence Survival

Full retraining from scratch is infeasible with only 40 annotated forget-set images, and our goal is to measure forgetting rather than simply apply it, so we fine-tune four independent copies of the poisoned detector on the unlearn set with every annotation replaced by an empty label set –

that is, the training signal tells the classification head that nothing at all should be detected in these images, driving down confidence specifically wherever the model previously fired, including at the poison location. Only classification-related parameters (`cls_score`, `cls_subnet`) are left trainable; all regression, backbone, and FPN parameters stay frozen, so ensemble members can shift classification confidence but cannot alter box geometry. The four members differ in learning rate, iteration count, and random seed:

Ensemble member	Learning rate	Iterations	Seed
<code>ens_cls_lr3e-4_it20</code>	3×10^{-4}	20	42
<code>ens_cls_lr5e-4_it20</code>	5×10^{-4}	20	42
<code>ens_cls_lr5e-4_it30_s7</code>	5×10^{-4}	30	7
<code>ens_cls_lr7e-4_it15_s13</code>	7×10^{-4}	15	13

Each member is run at a low inference threshold (`ENS_THRESH` = 0.01) so that even heavily suppressed poison boxes still come back with a small residual score.

For candidate box b_i with poisoned-model score s_i , and for each ensemble member m , its geometric match $e_{i,m}$ among that member’s predictions is found via $\text{IoU} \geq \text{MATCH_IOU} = 0.70$ – a strict threshold, appropriate since only classification parameters are trained and box geometry should be essentially unchanged. The per-member survival ratio is

$$r_{i,m} = \text{clip} \left(\frac{s_{e_{i,m}}}{\max(s_i, 10^{-6})}, 0, 1 \right), \quad (1)$$

set to 0 when no match exists (i.e., the box disappears entirely under unlearning). The final survival-derived poison score for box i is the ensemble-averaged confidence loss:

$$s_{\text{diff}}(i) = 1 - \frac{1}{4} \sum_{m=1}^4 r_{i,m}. \quad (2)$$

Intuitively, $s_{\text{diff}}(i) \rightarrow 1$ when a box’s confidence collapses across all four independently seeded unlearned classifiers – strong evidence of a poison-driven detection – and $s_{\text{diff}}(i) \rightarrow 0$ when confidence is essentially unaffected by unlearning, which is evidence of a genuine, well-supported streak.

4.4 Geometry and Appearance Priors

Geometry prior. From the 20 annotated poison boxes in the unlearn set we compute the mean and covariance of log-dimensions $\log(w, h)$, giving $\mu \in \mathbb{R}^2$ and $\Sigma \in \mathbb{R}^{2 \times 2}$ (regularised by $10^{-3}I$), and define a Gaussian-shaped geometry score

$$s_{\text{geo}}(b) = \exp \left(-\frac{1}{4} d(b)^\top \Sigma^{-1} d(b) \right), \quad d(b) = \log(w_b, h_b) - \mu, \quad (3)$$

i.e., an exponentiated (halved) squared Mahalanobis distance between a candidate box’s log-dimensions and the empirical poison-box geometry distribution. Exploratory analysis (Section 6) showed that poison-box geometry sits well inside the cloud of genuine streak-candidate geometries, giving this signal little separating power on its own; its fusion weight is accordingly demoted (`W_GEO` = 0.05).

Appearance prior. A compact convolutional network (TinyCNN) is trained on 64×64 , median/MAD-normalised crops around each candidate box, with positive examples drawn from the 20 true poison patches (augmented with reflections and small jitter) and negative examples

synthesised procedurally – randomly oriented, Gaussian-blurred line segments of varying length and contrast, pasted onto genuine background patches – so the classifier learns to separate poison appearance from any plausible streak rather than from a specific background. The resulting classifier reaches a validation accuracy of 0.800 on held-out synthetic negatives, and its sigmoid output on a candidate crop is used directly as $s_{\text{app}}(b) \in [0, 1]$.

4.5 Score Fusion

The three per-box signals are combined into a single poison-probability estimate

$$p_{\text{poison}}(i) = W_{\text{diff}} s_{\text{diff}}(i) + W_{\text{geo}} s_{\text{geo}}(i) + W_{\text{app}} s_{\text{app}}(i), \quad (4)$$

with default weights $W_{\text{diff}} = 0.75$, $W_{\text{geo}} = 0.05$, $W_{\text{app}} = 0.20$ (summing to 1), reflecting the empirical dominance of the ensemble-survival signal established in Section 6. The grid search in Section 4.7 additionally searches $W_{\text{diff}} \in \{0.6, 0.75, 0.9\}$ jointly with the remap parameters, selecting $W_{\text{diff}} = 0.6$ at the calibration optimum (Table 3).

4.6 Confidence Remapping

Given a candidate’s raw score s and fused poison probability p , the submitted confidence is produced by a piecewise remap function $R(s, p; \Theta)$, parameterised by $\Theta = (P_{\text{HI}}, P_{\text{LO}}, S_{\text{CONF}}, C_{\text{MAX}}, \varepsilon, \text{MIN_KEEP}, \text{BOOST})$:

$$R(s, p; \Theta) = \begin{cases} 0 & \text{if } s < \text{MIN_KEEP} \quad (\text{drop low-confidence noise}) \\ \varepsilon & \text{if } p \geq P_{\text{HI}} \quad (\text{flagged poison: demote, do not delete}) \\ C_{\text{MAX}} & \text{if } p \leq P_{\text{LO}}, \text{ BOOST}, s \geq S_{\text{CONF}} \\ s & \text{if } p \leq P_{\text{LO}} \quad (\text{trust the base detector}) \\ \max\left(\varepsilon, s \left(1 - \frac{p - P_{\text{LO}}}{P_{\text{HI}} - P_{\text{LO}}}\right)\right) & \text{otherwise (linear demotion band)} \end{cases} \quad (5)$$

Two hand-set presets bound the design space explored by the grid search: a conservative preset ($P_{\text{HI}} = 0.65$, $P_{\text{LO}} = 0.30$, $\text{MIN_KEEP} = 0.20$, $\text{BOOST} = \text{False}$) that prioritises poison suppression, and an aggressive preset ($P_{\text{HI}} = 0.50$, $\text{MIN_KEEP} = 0.12$) that keeps more low-confidence candidates at the cost of a higher expected poison leak. Both presets share $S_{\text{CONF}} = 0.50$, $C_{\text{MAX}} = 0.95$, $\varepsilon = 0.01$.

After remapping, an additional non-maximum-style cleanup step removes any surviving ε -demoted box that overlaps ($\text{IoU} \geq 0.2$) a retained higher-confidence box, preventing duplicate near-zero-confidence entries from inflating the submission.

4.7 Calibration Set and Grid Search

Because the test set has no ground truth, remap parameters cannot be tuned against it directly. Instead, we build a synthetic calibration set of 60 composite images, each constructed from a randomly sampled unlearn-set background augmented with (a) any native poison boxes already present, (b) one or two procedurally synthesised streaks (Section 4.4) labelled as genuine, and (c) zero to two transplanted poison patches (Section 4.2) labelled as poison. This produces a calibration set with a known poison/genuine label for every ground-truth box, so the maCADD metric (Section 4.8), including its asymmetric poison-aware penalty term, can be computed exactly.

A grid search sweeps $W_{\text{diff}} \in \{0.6, 0.75, 0.9\}$, $P_{\text{HI}} \in \{0.40, 0.50, 0.60, 0.70\}$, $\text{MIN_KEEP} \in \{0.08, 0.12, 0.20\}$, and $\text{BOOST} \in \{\text{True}, \text{False}\}$ (with P_{LO} , S_{CONF} , C_{MAX} , ε held fixed), evaluating calibration-set maCADD for every configuration and selecting the argmin as the tuned configuration (Table 3).

4.8 Evaluation Metric: maCADD

We define the mean asymmetric-confidence absolute difference (maCADD) between a reference detection set and a candidate submission. For a single image, reference boxes/scores (B_c, S_c) (optionally flagged as poison via a boolean mask) are matched to submitted boxes/scores (B_p, S_p) at IoU threshold t via the Hungarian algorithm [10], implemented through `scipy.optimize.linear_sum_assignment` [11]. For each matched pair (i, j) with score difference $d = s_{c,i} - s_{p,j}$, the cost contribution is

$$\text{cost}(i, j) = \begin{cases} d/A & \text{if reference box } i \text{ is poison and } d > 0 \quad (\text{under-penalise correctly demoted poison}) \\ -d & \text{if reference box } i \text{ is poison and } d \leq 0 \\ d & \text{if reference box } i \text{ is genuine and } d > 0 \\ -d/A & \text{if reference box } i \text{ is genuine and } d \leq 0 \quad (\text{under-penalise conservative demotion}) \end{cases} \quad (6)$$

with asymmetry factor $A = 10$. Unmatched reference and submitted scores are added to the cost in full – an unmatched confident detection is penalised as heavily as a large score disagreement. Per-image cost at threshold t is ACADD_t ; the full metric averages over $t \in \{0.2, 0.3, \dots, 0.9\}$ (nine thresholds), weighted proportionally to t itself so agreement at stricter IoU thresholds counts more, and finally averages over all images:

$$\text{maCADD} = \frac{1}{N} \sum_{\text{images}} \sum_t \frac{t}{\sum_{t'} t'} \cdot \text{ACADD}_t. \quad (7)$$

Lower maCADD indicates closer agreement with the trusted reference, with 0 indicating a perfect match. The metric was unit-tested against hand-computed cases prior to use – e.g., an unmatched reference score of 0.6 against an empty submission yields $\text{maCADD} = 0.6$, and a genuine box demoted from 1.0 to 0.2 at full match yields $\text{maCADD} = 0.04$ after the $/A$ discount.

Because ground truth is unavailable on the 4,000-image test set, a surrogate reference is built by taking the poisoned model’s own candidate output and removing every box whose fused poison probability exceeds a fixed flag threshold ($p_{\text{poison}} \geq 0.6$). This surrogate is not a substitute for true clean-model output, but it gives an internally consistent, self-referential yardstick for comparing remap variants under identical conditions; its use is limited strictly to relative comparison between candidate submissions and should not be read as an absolute correctness claim.

5 Experimental Setup

All experiments ran on a single workstation (Apple Silicon, MPS backend for the appearance classifier; CPU inference for Detectron2, since the deployed PyTorch/Detectron2 combination targets CPU execution) using PyTorch 2.8.0 and Detectron2 0.6. The base detection configuration is `COCO-Detection/retinanet_R_50_FPN` with `NUM_CLASSES = 1`, image size 1024×1024 , and batch size 4 during the short classification-head fine-tuning runs. The global seed is fixed at 42 throughout, with per-ensemble-member seed overrides of 7 and 13 as noted in Section 4.3. Candidate generation uses `CAND_THRESH = 0.05`; ensemble inference uses `ENS_THRESH = 0.01`; the geometric match threshold for ensemble comparison is `MATCH_IOU = 0.70`.

6 Results

6.1 Candidate Statistics and Trigger Locality

Candidate generation at the 0.05 threshold produced 8,299 boxes on the 4,000-image test set (4.15 boxes/image). The patch-transplantation probe (Section 4.2) recorded a firing rate of 99.00% (99/100 transplants), with a mean confidence of 0.585 at firing – comfortably above the 0.5 locality gate – confirming that the trigger is a portable, context-independent visual pattern and enabling full local grid-search calibration (Section 4.7).

6.2 Poison-Probability Signal Quality

Table 1 bins all 8,299 test-set candidates by raw poisoned-model score and reports the mean fused poison probability, along with the fraction of boxes exceeding $p_{\text{poison}} \geq 0.5$ and $p_{\text{poison}} \geq 0.7$ in each bin.

Table 1: Fused poison probability by raw candidate score bin ($N = 8,299$ candidates).

Score bin	n	Mean p_{poison}	Frac. $p \geq 0.5$	Frac. $p \geq 0.7$
[0.05, 0.10)	4300	0.6446	0.8716	0.3591
[0.10, 0.20)	1406	0.6542	0.8570	0.4182
[0.20, 0.40)	800	0.6846	0.8688	0.5688
[0.40, 0.70)	1100	0.6856	0.8809	0.5691
[0.70, 1.01)	693	0.5987	0.7749	0.2713

As a sanity check, p_{poison} was computed on the 20 candidate detections independently matched ($\text{IoU} \geq 0.3$) to the known-poison ground-truth boxes in the unlearn set: minimum 0.459, median 0.714, maximum 0.841. This distribution directly shaped the calibration search space – the demotion threshold P_{HI} has to sit below the bulk of it to avoid leaking poison mass, which ruled out earlier, higher threshold choices (e.g., 0.85) that would sit above most true-poison scores.

6.3 Grid Search and Selected Calibration

Table 2 summarises the top calibration-set configurations from the grid search (lowest calibration maCADD first, among the recorded top-10).

Table 2: Top calibration-set grid-search results (subset; lower maCADD is better).

W_{diff}	P_{HI}	P_{LO}	MIN_KEEP	BOOST	S_{CONF}	C_{MAX}	maCADD
0.6	0.6	0.3	0.20	False	0.5	0.95	0.6981
0.6	0.6	0.3	0.20	True	0.5	0.95	0.6992
0.6	0.7	0.3	0.20	False	0.5	0.95	0.7154
0.6	0.7	0.3	0.20	True	0.5	0.95	0.7165
0.6	0.6	0.3	0.12	False	0.5	0.95	0.7212

The selected tuned configuration was $\Theta_{\text{tuned}} = \{P_{\text{HI}} = 0.6, P_{\text{LO}} = 0.3, \text{MIN_KEEP} = 0.2, \text{BOOST} = \text{False}, S_{\text{CONF}} = 0.5, C_{\text{MAX}} = 0.95, \varepsilon = 0.01\}$ with $W_{\text{diff}} = 0.6$ (Table 3).

Table 3: Selected calibration parameters for the tuned remap variant.

Parameter	Value
W_{diff}	0.6
P_{HI}	0.6
P_{LO}	0.3
MIN_KEEP	0.2
BOOST	False
S_{CONF}	0.5
C_{MAX}	0.95
ε	0.01

6.4 Poison Mass Suppression

Table 4 reports total submitted confidence mass on the unlearn set, restricted to boxes matched ($\text{IoU} \geq 0.3$) to the 20 known poison ground-truth boxes, for the raw poisoned model and each of the three remap variants. An empirical floor of roughly 0.2 reflects the residual mass expected from applying the ε -demotion policy to 20 flagged boxes.

Table 4: Poison confidence mass on the 20 known poison boxes (unlearn set), before and after remapping.

Variant	Poison mass
Poisoned model (raw)	11.72
Conservative	0.98
Aggressive	0.21
Tuned	0.39
Theoretical floor	≈ 0.20

The aggressive variant comes closest to the theoretical floor, but Table 6 shows this costs substantially more submitted boxes overall (and thus more false-positive risk), while the conservative variant leaves more residual poison mass than aggressive but achieves the best surrogate maCADD (Section 6.5) – indicating a better overall confidence-fidelity trade-off once genuine-detection preservation is taken into account.

6.5 Surrogate maCADD and Final Selection

Table 5 reports surrogate maCADD (Section 4.8) on the 4,000-image test set for the raw poisoned output and each remap variant.

Table 6 shows the number of boxes retained in each final submission file, alongside the raw 0.2-threshold poisoned baseline.

Despite the three remap variants and the additionally grid-searched configuration, the conservative preset achieved the lowest surrogate maCADD (0.1353) and was chosen as the final submission, retaining a box count (2,586) close to the raw-poisoned baseline (2,593) while suppressing over 91% of poison confidence mass (Table 4) relative to the raw model.

Table 5: Surrogate maCADD on the test set (lower is better). Reduction is relative to raw poisoned output.

Variant	Surrogate maCADD	Relative reduction
Raw poisoned	0.7004	—
Conservative	0.1353	80.7%
Tuned	0.1514	78.4%
Aggressive	0.1986	71.6%

Table 6: Submitted box counts per variant, test set ($N = 4,000$ images).

Submission	Boxes
Raw poisoned @ 0.2	2593
Conservative	2586
Aggressive	3506
Tuned	2588

7 Discussion

7.1 Why Confidence Survival Dominates the Fusion

The final weighting ($W_{\text{diff}} = 0.6\text{--}0.75 \gg W_{\text{geo}} = 0.05$) follows directly from the geometry exploratory analysis: poison-box log-dimensions fall well within the bulk of the genuine streak-candidate geometry distribution, so the geometry term contributes little separating information relative to the ensemble-survival signal, which directly measures the causal effect of classification-head unlearning on each box’s own confidence. This lines up with the backdoor literature’s general finding that trigger-driven behaviour is a property of learned decision boundaries rather than of superficial geometric statistics [16, 17], and supports framing confidence-survival as a detection-time analogue of pruning-based defenses [15].

7.2 The Role of the Transplantation Gate

The 99% transplant firing rate is strong evidence that the poison trigger in this model is not scene-contextual, which is exactly the condition under which local, data-driven calibration (Section 4.7) can be trusted. Had the firing rate fallen below the 0.5 gate, the design would have fallen back to fixed conservative presets instead of a tuned grid search, since a contextual trigger would make transplant-based and forget-set-based calibration signals unreliable proxies for test-set behaviour. This gating logic generalises: any unlearning-by-fine-tuning pipeline built on a small forget set should first check that the forget set’s trigger instances are representative of how the trigger shows up elsewhere, which echoes existing concerns about the generalisation of small-sample backdoor defenses [16, 18].

7.3 Conservative vs. Tuned vs. Aggressive

It’s notable that the grid-searched tuned configuration did not outperform the hand-set conservative preset on the test-set surrogate metric, despite winning on the calibration set by construction. This is a textbook symptom of calibration-set/test-set distribution shift: the 60-image synthetic

calibration set, built from procedurally pasted patches and synthetic streaks, cannot perfectly capture the true diversity of streak morphology and poison manifestation across 4,000 real test images. We treat this as a useful negative result – automatic hyperparameter selection under a small, synthetic calibration budget should be validated, not blindly trusted, against a broader (even if imperfect) surrogate signal before deployment.

7.4 Limitations

Several limitations qualify these results. First, and most importantly, the surrogate maCADD reference is self-referential: it is derived from the same poisoned model’s own output, filtered by the very poison-probability score being evaluated, and is therefore susceptible to optimistic bias if a remap variant simply agrees with the fusion score rather than with true clean behaviour. Second, the appearance classifier’s 0.800 validation accuracy is measured against synthetic negative streaks rather than the full diversity of real astronomical clutter (cosmic-ray hits, satellite glints, diffraction spikes), and may not generalise as cleanly to test-set edge cases. Third, the ensemble unlearning procedure freezes all non-classification parameters; if the poisoning also affected box-regression behaviour, this pipeline would not detect or correct it, since `MATCH_IOU = 0.70` was chosen specifically on the assumption of near-identical geometry across ensemble members. Fourth, the 20-exemplar forget set is small, and the four ensemble members – while independently seeded – share the same architecture, base weights, and fine-tuning recipe family, which limits the diversity of the “unlearning” perspectives being ensembled.

8 Conclusion and Future Work

We presented a complete diagnostic-and-repair pipeline for a poisoned RetinaNet streak detector, combining a trigger-locality probe, an ensemble of classification-head-only unlearned models, a Mahalanobis geometry prior, a lightweight appearance classifier, and a grid-searched confidence-remapping policy, evaluated with a purpose-built asymmetric metric (maCADD). On a 4,000-image test set, the selected conservative remap variant reduced surrogate maCADD from 0.7004 to 0.1353 (an 80.7% relative reduction) while suppressing over 91% of confidence mass on known poison exemplars, at a cost of only 7 fewer submitted boxes than the raw poisoned baseline out of 2,593. These results suggest that ensemble-based confidence-survival signals, derived from cheap, classification-head-only unlearning on a small forget set, can meaningfully approximate the effect of full backdoor removal in detection settings where clean reference weights are unavailable.

Future work includes: (i) replacing the self-referential surrogate metric with an externally validated clean-model reference where one is available; (ii) extending the unlearning ensemble to include architecturally diverse members (e.g., a parallel Faster R-CNN branch [5]) to reduce shared-bias risk; (iii) exploring certified or exact unlearning frameworks such as SISA [22] adapted to the small-forget-set, single-checkpoint setting; and (iv) evaluating the pipeline’s robustness against adaptive attackers aware of the transplantation-gate and survival-scoring defense, in the spirit of adaptive-adversary evaluations common in the broader adversarial-robustness literature [30].

Data and Code Availability

The detector configuration, calibration procedure, and evaluation code described in this paper correspond directly to the released implementation accompanying the “Neural Debris Removal in

Streak Detection Models” dataset (unlearn set: 40 images; test set: 4,000 images). All numerical results reported in Section 6 are taken directly from this implementation’s output logs.

References

- [1] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [2] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick, “Detectron2,” GitHub repository, Facebook AI Research, 2019. <https://github.com/facebookresearch/detectron2>
- [3] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [5] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.
- [6] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common Objects in Context,” in *Proc. European Conf. on Computer Vision (ECCV)*, 2014, pp. 740–755.
- [7] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The Pascal Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.
- [8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012.
- [9] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [10] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [11] P. Virtanen et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [12] T. Gu, B. Dolan-Gavitt, and S. Garg, “BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain,” *arXiv preprint arXiv:1708.06733*, 2017.
- [13] X. Chen, C. Liu, B. Li, K. Lu, and D. Song, “Targeted Backdoor Attacks on Deep Learning Systems Using Data Poisoning,” *arXiv preprint arXiv:1712.05526*, 2017.
- [14] Y. Liu, S. Ma, Y. Aafer, W.-C. Lee, J. Zhai, W. Wang, and X. Zhang, “Trojaning Attack on Neural Networks,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2018.

- [15] K. Liu, B. Dolan-Gavitt, and S. Garg, “Fine-Pruning: Defending Against Backdooring Attacks on Deep Neural Networks,” in *Proc. Int. Symposium on Research in Attacks, Intrusions, and Defenses (RAID)*, 2018, pp. 273–294.
- [16] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao, “Neural Cleanse: Identifying and Mitigating Backdoor Attacks in Neural Networks,” in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2019, pp. 707–723.
- [17] B. Tran, J. Li, and A. Madry, “Spectral Signatures in Backdoor Attacks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.
- [18] B. Chen, W. Carvalho, N. Baracaldo, H. Ludwig, B. Edwards, T. Lee, I. Molloy, and B. Srivastava, “Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering,” *arXiv preprint arXiv:1811.03728*, 2018.
- [19] Y. Li, B. Wu, Y. Jiang, Z. Li, and S.-T. Xia, “Backdoor Learning: A Survey,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 1, pp. 5–22, 2024.
- [20] M. Goldblum, D. Tsipras, C. Xie, X. Chen, A. Schwarzschild, D. Song, A. Madry, B. Li, and T. Goldstein, “Dataset Security for Machine Learning: Data Poisoning, Backdoor Attacks, and Defenses,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 2, pp. 1563–1580, 2023.
- [21] Y. Cao and J. Yang, “Towards Making Systems Forget with Machine Unlearning,” in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2015, pp. 463–480.
- [22] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, “Machine Unlearning,” in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2021, pp. 141–159.
- [23] A. Golatkar, A. Achille, and S. Soatto, “Eternal Sunshine of the Spotless Net: Selective Forgetting in Deep Networks,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 9304–9312.
- [24] T. T. Nguyen, T. T. Huynh, P. L. Nguyen, A. W.-C. Liew, H. Yin, and Q. V. H. Nguyen, “A Survey of Machine Unlearning,” *arXiv preprint arXiv:2209.02299*, 2022.
- [25] K. Kurita, P. Michel, and G. Neubig, “Weight Poisoning Attacks on Pretrained Models,” in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 2793–2806.
- [26] D. A. Duev, A. Mahabal, Q. Ye, K. Tirumala, J. Belicki, R. Dekany, S. Frederick, M. J. Graham, G. Helou, R. R. Laher, F. J. Masci, T. A. Prince, R. Riddle, P. Rosnet, and M. T. Soumagnac, “DeepStreaks: Identifying Fast-Moving Objects in the Zwicky Transient Facility Data with Deep Learning,” *Monthly Notices of the Royal Astronomical Society*, vol. 486, no. 3, pp. 4158–4165, 2019.
- [27] P. Jia, Q. Liu, and Y. Sun, “Detection and Classification of Astronomical Targets with Deep Neural Networks in Wide-Field Small Aperture Telescopes,” *The Astronomical Journal*, vol. 159, no. 5, p. 212, 2020.

- [28] E. C. Bellm, S. R. Kulkarni, M. J. Graham, et al., “The Zwicky Transient Facility: System Overview, Performance, and First Results,” *Publications of the Astronomical Society of the Pacific*, vol. 131, no. 995, p. 018002, 2019.
- [29] L. Varela, L. E. Boucheron, N. Malone, and N. Spurlock, “Streak Detection in Wide Field of View Images Using Convolutional Neural Networks,” in *Proc. Advanced Maui Optical and Space Surveillance Technologies Conference (AMOS)*, 2019.
- [30] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, “Intriguing Properties of Neural Networks,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2014.